

Full length article

A classification-by-retrieval framework for few-shot anomaly detection to detect API injection

Udi Aharon^{a,b}*, Ran Dubin^{a,b}, Amit Dvir^{a,b}, Chen Hajaj^{d,b,c}

^a Department of Computer and Software Engineering, Golan Heights 1, Ariel, 4077625, Israel

^b Ariel Cyber Innovation Center, Ariel University, Golan Heights 1, Ariel, 4077625, Israel

^c Data Science and Artificial Intelligence Research Center, Ariel University, Golan Heights 1, Ariel, 4077625, Israel

^d Department of Industrial Engineering & Management, Ariel University, Golan Heights 1, Ariel, 4077625, Israel

ARTICLE INFO

Keywords:

API security
Anomaly detection
Few-shot learning
Classification-by-retrieval
ANN
NLP

ABSTRACT

Application Programming Interface (API) Injection attacks refer to the unauthorized or malicious use of APIs, which are often exploited to gain access to sensitive data or manipulate online systems for illicit purposes. Identifying actors that deceitfully utilize an API poses a demanding problem. Although there have been notable advancements and contributions in the field of API security, there remains a significant challenge when dealing with attackers who use novel approaches that do not match the well-known payloads commonly seen in attacks. Also, attackers may exploit standard functionalities unconventionally and with objectives surpassing their intended boundaries. Thus, API security needs to be more sophisticated and dynamic than ever, with advanced computational intelligence methods, such as machine learning models that can quickly identify and respond to abnormal behavior. In response to these challenges, we propose a novel unsupervised few-shot anomaly detection framework composed of two main parts: First, we train a dedicated generic language model for API based on FastText embedding. Next, we use Approximate Nearest Neighbor search in a classification-by-retrieval approach. Our framework allows for training a fast, lightweight classification model using only a few examples of normal API requests. We evaluated the performance of our framework using the CSIC 2010 and ATRDF 2023 datasets. The results demonstrate that our framework improves API attack detection accuracy compared to the state-of-the-art (SOTA) unsupervised anomaly detection baselines.

1. Introduction

Application Programming Interface (API) refers to a set of routines, procedures, resources, and protocols that permit the interaction between software systems and data exchange services (Balsari et al., 2018; Ofoeda et al., 2019; Mendoza and Gu, 2018). APIs are an evolving technology for orchestrating applications utilizing web technology (Benzaid and Taleb, 2020; IBM, 2016). Recently, it has been argued that we live in an API economy (IBM, 2016) due to the growing interconnectedness of people, applications, and systems, all of which are powered by APIs. These interfaces now serve as the foundational framework of the digital ecosystem, establishing connections between industries and economies to foster value creation and cultivate innovative capabilities (Ofoeda et al., 2019). APIs find utilization across

a broad spectrum of services, including Web applications, Operation Systems (OS), Databases, and Hardware (Reddy, 2011). The increasing ease of building web applications has led to a rise in agile development, where even inexperienced engineers can deploy applications. However, this approach often lacks strong security design or hardening planning. This can lead to vulnerabilities in application logic and inadequate consideration of security impacts. For instance, failure to properly constrain resources or access levels could lead to denial of service attacks (Sun et al., 2021). As a result, the extensive adoption of web APIs has heightened the potential of user safety and privacy breaches, making APIs a prime target for cyber attackers (Mendoza and Gu, 2018). In recent years, there have been several high-profile

* Correspondence to: Department of Computer Science, Ariel University, Golan Heights 1, 4077625, Ariel, Israel.

E-mail addresses: udi.aharon@msmail.ariel.ac.il (U. Aharon), rand@ariel.ac.il (R. Dubin), amitdv@ariel.ac.il (A. Dvir), chenha@ariel.ac.il (C. Hajaj).

API attacks (Coyne, 2020; BBC, 2019; Greig, 2023; Tara Siegel et al., 2017), such as the Zoom video conferencing platform in 2020.

To address these challenges, the Open Web Application Security Project (OWASP¹) provides resources, tools, and best practices to help organizations and developers enhance the security of their web applications and protect against malicious attacks. One of the most well-known contributions is the OWASP API Top 10, which outlines the ten most critical API security risks (Fett et al., 2016). Injection flaws enable attackers to inject malicious code through an application into another system. These flaws encompass a variety of attacks, such as Directory Traversal, SQL Injection (SQLi), LDAP Injection, and Cross-Site Scripting (XSS). OWASP classifies Injection flaws under the broader category of Unsafe Consumption of APIs security risks.²

Despite the progress made in prior research on utilizing machine learning and deep learning models for protecting APIs against both known and unknown attacks (Chan et al., 2023; Baye et al., 2021; Harlicaj et al., 2021), there are lingering concerns that remain unresolved. These concerns encompass several challenges, such as effectively detecting zero-day vulnerabilities, minimizing false positives, and addressing real-time and continuous protection requirements. Given that a zero-day API attack involves an unknown vulnerability that the security solutions, such as web application firewalls, are unaware of, it becomes necessary to employ few-shot learning techniques. The anomaly detection model needs to leverage its comprehension of previously encountered samples to make predictions on new, unseen samples. In light of the requirement to improve traditional methods for training binary anomaly classifiers, the utilization of Classification-by-Retrieval presents a solution (Shen et al., 2017). This approach enables the construction of neural network-based classifiers without requiring computationally intensive training procedures. Consequently, it facilitates the development of a lightweight model that can be trained with minimal examples per class or even a model capable of classifying multiple classes (Shi et al., 2022; Qamar et al., 2008; Valero-Mas et al., 2023). In this context, we propose a novel unsupervised few-shot anomaly detection framework, utilizing FastText embedding and Approximate Nearest Neighbor search (FT-ANN). This framework offers a unique representation for the API request, facilitating intelligent segmentation based on the API endpoint. The term “endpoint” refers to any data or metadata that may represent a type, an origin, or identification of a respective API request. Endpoint is defined as the combination of the method, host, and path (Battle and Benson, 2008). This paper’s contributions are itemized as follows:

- We introduce a novel unsupervised few-shot anomaly detection framework, utilizing FastText embedding and ANN search, which leverages a Classification-by-retrieval approach. This framework employs a Classification-by-retrieval approach and allows for training a single retrieval model that can handle multiple base-lines concurrently, such as API endpoints or multiple API domains. Our approach provides a more efficient and scalable solution for anomaly detection by reducing the number of required models and supporting incremental index updates. This is in contrast to traditional models, which usually require the entire dataset to be present for training new examples. Furthermore, the high performance and speed of the in-memory similarity search, widely adopted in many ecosystems due to its flexible nature, significantly enhance the overall efficiency of our framework.
- We define a novel tokenizer that specifically emphasizes the language factors present in APIs, addressing the unique challenges associated with API-based natural language processing. APIs are defined by a URL-based syntax in which each URL corresponds

Table 1

List of abbreviations.

Abbreviation	Definition
AE	Autoencoder
ANN	Approximate Nearest Neighbor
API	Application Programming Interface
CBLOF	Clustering-Based Local Outlier Factor
CNN	Convolutional Neural Network
DeepSVDD	Deep Support Vector Data Description
FB	Feature Bagging
GRU	Gated Recurrent Unit
HBOS	Histogram-based Outlier Detection
IF	Isolation Forest
KDE	Kernel Density Estimation
LMDD	Linear Model Deviation-based Outlier Detection
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
MCD	Minimum Covariance Determinant
NLP	Natural Language Processing
OCSVM	One-class SVM
PCA	Principal Component Analysis
QPS	Queries per second
RDA	Regularized Deep Autoencoder
RNN	Recurrent Neural Networks
SOTA	State-of-the-art
SQLi	SQL Injection
SVM	Support Vector Machine
XSS	Cross-Site Scripting

to a particular resource or action, and they include fundamental actions such as GET and POST, determining the structure of requests and responses. Additionally, APIs utilize the standard structure of HTTP headers to transmit metadata pertaining to both the request and the response. Unlike existing tokenizers, our approach considers these specific language characteristics of APIs, enabling more accurate and efficient processing of API-related text.

- Our language model is designed to be domain-agnostic, eliminating the need for retraining when transitioning to different API domains. This flexibility allows our model to seamlessly serve various domains without sacrificing performance or requiring additional training efforts.
- Our model’s agnostic nature allows it to seamlessly adapt and address the unique requirements and challenges, as also outlined in the OWASP Top 10 API vulnerabilities, posed by different API forms, including REST, GraphQL, gRPC, and WebSockets, regardless of the emphasis on HTTP datasets during the demonstration.

The remaining sections of this paper are structured in the following manner. In Section 2 we provide an overview of the relevant literature; The architecture of the FT-ANN framework is outlined in Section 3; Section 4 describes the datasets that were used for this paper; The experimental design is outlined in Section 5; Section 6 outlines the experimental results, which focus on measuring the effectiveness the FT-ANN framework and presenting experimental results compared to various state-of-the-art benchmarks; Finally, we draw conclusions, current research limitations, and suggestions for future work in Section 7. For ease of reading, we provide a list of common abbreviations in Table 1.

2. Related work

This section reviews relevant research on API and web traffic classification, focusing on the use of machine learning, deep learning, and natural language processing (NLP) techniques. A summary of these studies is presented in Table 2.

Reddy and Rudra (2021) proposed supervised sequence models based on Recurrent Neural Networks (RNN) to identify malicious injections in API requests in addition to a heuristic rule that classified

¹ <https://owasp.org/>

² <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/>

10% of each request sequence with a probability of 60% as valid to minimize the number of false positives. However, such rules can introduce subjective biases and limitations. They generated a custom-labeled dataset but used only the request payloads. This presents a significant constraint in real-world scenarios, where malicious actors could potentially manipulate a majority of the request components, including headers (Ombagi, 2017). They compared six unidirectional and bidirectional RNN models by evaluating various performance measures and showed a 50% decrease in false positive cases. Jemal et al. (2021a) suggested the Convolutional Neural Network (CNN) method to detect web attacks. They concluded that appropriately adjusting hyper-parameters and employing a data pre-processing approach significantly impacts the detection rate. Jemal et al. (2020) propose a supervised Memory CNN that combines a CNN and Long Short-Term Memory (LSTM) to identify patterns of malicious requests within sequences of requests. Niu and Li (2020) propose a technique for detecting web attacks based on a supervised CNN and Gated Recurrent Unit (GRU). They extracted statistical features and utilized a Word2Vec model to extract word embeddings, resulting in a 3-dimensional input for the suggested CNN-GRU method. A fully connected layer was used for classification. Yu et al. (2020) combined a CNN and Support Vector Machine (SVM) to detect malicious web server requests. Baye et al. (2021) also utilized SVM with a Linear Kernel as a two-class classifier to identify anomalous API requests. To form a training dataset that accurately represented authentic API logs, they employed a technique for outlier detection based on Gaussian Distribution, generating a synthetic dataset with labeled examples.

Gniewkowski et al. (2021) suggested an NLP-based self-supervised anomaly detection methodology that employs RoBERTa for embedding HTTP requests. A notable limitation in their method is the relatively long training and inference time, which presents a considerable drawback in real-time systems. Moradi Vartouni et al. (2019) introduced an unsupervised anomaly detection technique utilizing Autoencoder (AE) and LSTM for feature extraction and Isolation Forest (IF) for classification. However, they faced limitations related to the choice of encoding method for HTTP data, the non-stationary nature of HTTP data, and the integration of multiple feature sets, which may impact the effectiveness of the proposed approach. In a more recent study (Vartouni et al., 2023), an unsupervised Deep Support Vector Data Description (DeepSVDD) method was proposed. They compared two feature extraction approaches, namely bigram (2-gram) and one-hot. They pointed out the limitation of lacking support for data streams in incremental learning, emphasizing the need for future research to address this aspect. Mac et al. (2018) also suggested employing AEs for unsupervised learning, evaluated various variants, and concluded that the Regularized Deep AE (RDA) performed the best. They processed the raw HTTP requests and tokenized the URLs by replacing the characters with their corresponding ASCII codes. Vartouni et al. (2018) used a Stacked AE to extract HTTP features and employed IF as the classifier. In a Stacked AE, the output of each AE serves as the input for the next layer. They demonstrated that using various activation functions and optimization techniques can improve the learning process. Demirel and Sandikkaya (2023) proposed combining the Local Outlier Factor (LOF) algorithm and an AE to enhance classification accuracy. The study evaluates different combination approaches, including the Behavior Knowledge Space (BKS) for combining multiple classifiers based on their predictions and Weighted Borda Counts (wBorda) for weighted voting systems, to assess their effectiveness. Karlsen et al. (2024) proposed integrating embeddings from Large Language Models (LLMs) such as GPT-2 and GPT-NEO into an AE. However, this method may be less suitable for real-time processing applications due to the complexities of fine-tuning LLMs and increased prediction times (see Table 2).

3. Framework

The analysis of API requests can be framed as a problem in NLP. One challenge lies in selecting a language model capable of generating a vector space representation. In our work, we decided to utilize the FastText (Research, 2017) model. FastText embeddings primarily aim to factor in the internal structure of words rather than simply learning word representations. This feature is especially advantageous for morphologically complex languages, allowing representations for various morphological forms of words to be learned separately. FastText offers Skip-gram and Continuous Bag-of-Words (CBOW) models to compute word representations. Although CBOW learns faster than Skip-gram, Skip-gram outperforms CBOW on small datasets (Bansal and Srivastava, 2018). FastText adopts a character n-gram approach to tokenize words, which effectively tackles Out-of-vocabulary (OOV) problems. This method not only generates embeddings for common words but also for rare, misspelled, or previously unseen words in the training corpus. In the realm of API security, it is essential to examine the internal structure of requests to comprehend their purpose and context. Security threats might be hidden within seemingly innocuous text, which could include the use of uncommon or previously unseen terms, or even efforts to mask their intention through spelling errors (Xiao et al., 2016). Moreover, APIs might be required to accommodate a diverse range of languages, including those characterized by intricate morphological structures (Seyyar et al., 2022).

Prior works have proposed various methods for detecting anomalies. While most of these approaches focus on classic classification tasks, we propose utilizing the ANN vector similarity method for identifying in-distribution records. The similarity search concept pertains to the process of identifying data points in a dataset that demonstrate similarities with a specific pattern, commonly referred to as a query (Chávez et al., 2001). To measure the similarity of a pair of data points, a distance function is used, where a small distance indicates that the two points are more similar or “closer” to each other (Ponomarenko et al., 2014). The Nearest Neighbor (NN) search is a specific type of similarity search used to identify data points nearest in distance to a provided query point (Indyk and Motwani, 1998). The ANN allows search despite the possibility of not retrieving all neighbors in a metric space (Indyk and Motwani, 1998).

While both ANN and traditional NN are rooted in the fundamental concept of similarity search, traditional NN search involves an exhaustive examination of all data points to find the NN, which can be prohibitively time-consuming for large datasets (Indyk and Motwani, 1998). In contrast, ANN employs techniques that trade off a slight loss in precision for substantial gains in speed, enabling the identification of ANN without examining every data point. This efficiency becomes especially crucial in real-world API applications where rapid response times are essential. Additionally, ANN methods are adaptable to high-dimensional spaces (Chávez et al., 2001; Hajebi et al., 2011), where traditional NN searches can suffer from the “curse of dimensionality”, in which the computational requirements for exact NN search become prohibitively high as the dimensionality of the dataset increases (Indyk and Motwani, 1998).

The framework proposed (depicted in Fig. 1) comprises a combination of a FastText embedding network and a retrieval layer, which includes an ANN matching component and a result aggregation component built upon it, forming the FT-ANN system. In Phase 1 (Fig. 1), we initiate the process by training a generic language model to serve as a reliable baseline for the detection model. To gather a substantial dataset, we crawled a random sample of websites from the Tranco top websites list, collecting over a million examples of normal web traffic. This language model now serves as a pre-trained baseline, applicable to any real-world API anomaly detection system.

Proceeding to Phase 2 (Fig. 1), the unsupervised detection model is trained, consisting of a pre-processing step and a Classification-by-retrieval framework. The pre-processing step validates the HTTP

Table 2
Overview of machine learning and NLP-based techniques for detecting attacks in HTTP traffic.

Ref	Task	Technique(s)	Method(s)	Dataset(s)
Yu et al. (2020)	Supervised	TextCNN, SVM	Word embedding, t-SNE, Statistical features	CSIC 2010
Niu and Li (2020)	Supervised	CNN, GRU	Word embedding, Statistical features	CSIC 2010
Jemal et al. (2020)	Supervised	CNN, LSTM	ASCII embedding	CSIC 2010
Reddy and Rudra (2021)	Supervised	RNN	Word embedding	Custom
Jemal et al. (2021a)	Supervised	CNN	Word embedding, Character embedding	CSIC 2010 CSIC 2010
Gniewkowski et al. (2021)	Self-supervised	RoBERTa	BBPE, Doc2vec	Custom
Mac et al. (2018)	Unsupervised	AE	ASCII embedding	CSE-CIC-IDS2018 CSIC 2010
Vartouni et al. (2018)	Unsupervised	SAE, IF	Character N-grams	CSIC 2010
Moradi Vartouni et al. (2019)	Unsupervised	AE LSTM	Character embedding	CSIC 2010
Vartouni et al. (2023)	Unsupervised	DeepSVDD	Character embedding	CSIC 2010 ECML/PKDD 2007 CSIC 2010
Demirel and Sandikkaya (2023)	Unsupervised	AE, LOF	ASCII embedding	Custom Fwaf CSIC 2010
Karlsen et al. (2024)	Unsupervised	AE, GPT-2, GPT-NEO	Sentence embedding	ECML/PKDD 2007 CSIC 2010
FT-ANN	Unsupervised	Classification-by-retrieval	Sentence embedding	ATRDF 2023

headers against the standard structure. Additionally, a unique data transformation is applied to simplify the API vocabulary. The detection model is trained solely on normal API traffic and constructs a single ANN model for all endpoints. For each endpoint, a threshold is calculated and utilized during the detection stage.

Finally, in Phase 3 (Fig. 1), the model's performance is validated by allowing an index search for every request. During validation, we standardize the request using the same pre-processing stages applied during the detection model training phase. The text is then transformed into a vector representation using our generic language model through inference. These embedding vectors are used in the ANN search, which returns the K-NNs from the specific endpoint index collection, followed by a maximum distance scaling layer. The final score obtained enables the model to ascertain whether the incoming API request is normal or an anomaly by comparing it with the pre-defined threshold for each API endpoint.

3.1. Data pre-processing

For both training and inferring the detection model, we employ a unique pre-processing technique consisting of three phases, as depicted in Fig. 1 in steps 2.1 and 3.1. First, decoding URL special characters, decompressing request body content, and converting every character within the request data string into lowercase. Then, validating request headers to ensure they are formatted correctly and extracting the endpoint definition, as depicted in Fig. 1, steps 2.1.1 and 3.1.1. API request headers typically provide information about the request context, supply authentication credentials, and provide information about the client (e.g., a person, a computing device, and/or a browser application) that had initiated the API (Buchanan et al., 2018). API request header fields are typically derived from a limited set of options. Accordingly, the data preparation also uses a fixed set of rules to validate the content of request headers and filter-out headers according to these rules. Headers that include valid or approved strings may be transferred to their destination as an API request and may be excluded from additional processing. For example, request headers may include host strings, which specify host or Internet Protocol (IP) addresses and/or port numbers of a server to which the API request is being sent. Valid IPv4 syntax should have the following format: $(0 \leq n < 256).(0 \leq n < 256).(0 \leq n < 256).(0 \leq n < 256) : (1 \leq n \leq 65535)$.

Lastly, as depicted in Fig. 1, steps 2.1.2 and 3.1.2 convert received requests into abstracted versions using a conversion schema. For instance, we replace non-numeric single characters with the string "chr",

which serves as a representative, abstract version of the original request string. Another example involves converting non-textual symbols into pre-defined textual strings. For instance, colons (":") may be converted to the string "colon". During the pre-processing stage, the API language has been refined to achieve optimally, enabling it to describe various transactions in a consistent format without sacrificing their original meanings. In fact, in most cases, these requests have been intelligently merged into a single text representation. This consolidation streamlines the data and ensures that the essential information pertaining to different transactions remains preserved. For illustration, an example of the data pre-processing steps is shown in Fig. 2(a).

3.2. Unsupervised ML language model

Our framework leverages FastText for unsupervised learning. FastText possesses the capability to encapsulate substantive knowledge about words while integrating morphology details, a crucial aspect of API attack detection. While deep learning models have excelled and commonly provide state-of-the-art results across various NLP tasks, to the best of our knowledge, no previous NLP pre-trained model on the API traffic domain has been publicly published. In the training phase of the method, we built a single generic FastText language model (based on Research (2017)) from scratch using the normal API traffic collected by crawling Tranco's list of the most popular websites, as can be seen in Fig. 1 Phase 1. For training the generic language model, we used the default hyper-parameters of Research (2017), which encompass a learning rate of 5%, a word vector size of 100, and a context window size of 5. The model was then trained for 5 epochs. We utilized the CBOW model as the dataset is relatively large, and CBOW embeddings are precise enough for anomaly detection and computed in a shorter time than skip-gram (Joulin et al., 2017). For training the detection model and inferring, we extract the vector representation of words for every input line.

3.3. ANN

We obtained ANN to identify the normal representation of an API endpoint. We train a single detection model, which is used to describe a normal representation of all endpoints. During the detection model training stage, each API request is represented as a vector in the textual embedding space, including endpoint information. We employ cosine distance to measure the similarity between data points as it has been applied in numerous text mining endeavors, such as text classification,

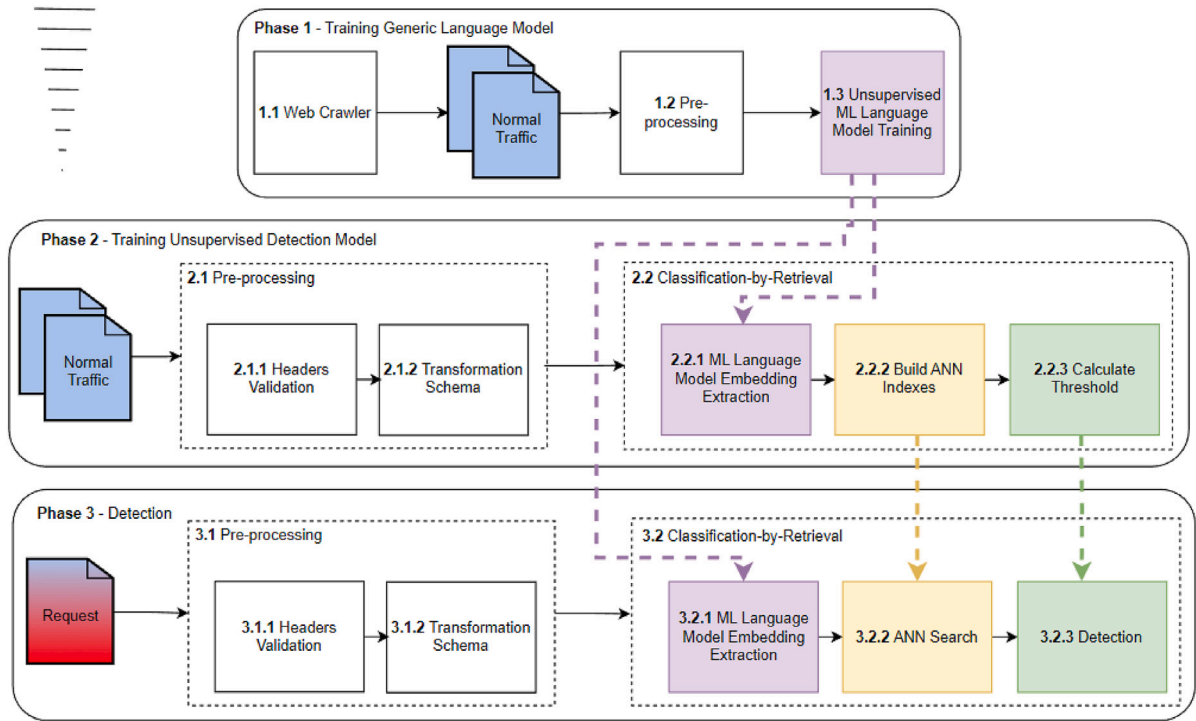


Fig. 1. The FT-ANN Framework: Training and Inference Methodology.

and information retrieval (Li and Han, 2013). Additionally, it has been proven to be effective for Out-of-Distribution (OOD) detection tasks (Techapanurak et al., 2020; Hsu et al., 2020). Cosine similarity is a widely used measure of similarity that calculates the angle formed by a pair of vectors. When measuring the similarity between two patterns, the Euclidean distance increases as they become less similar, while the cosine similarity increases as they become more similar. Unlike Euclidean distance, cosine similarity is unaffected by the magnitude of the vectors being compared (Xia et al., 2015). The embedding vector feeds the Hierarchical Navigable Small Worlds (HNSW) graph (Naidan et al., 2014) to build new indexes of data points. HNSW allows new elements to be inserted into the existing graph structure without requiring the reconstruction of the entire graph. This is achieved through a hierarchical structure, where each element is randomly assigned to different levels, and connections to neighbors are incrementally established using a greedy search process. This incremental construction of connections enables the HNSW graph to efficiently adapt to new data points, making it well-suited for incremental learning scenarios where updated embeddings can be continuously incorporated into the existing index without the need for complete re-computation (Malkov and Yashunin, 2018). During the detection stage, the model compares new API vectors of incoming API requests with API vectors of the same API endpoint information to evaluate the normality or anomaly of the incoming API request. The comparison is made by querying the similarity between the input vector to the closest k objects.

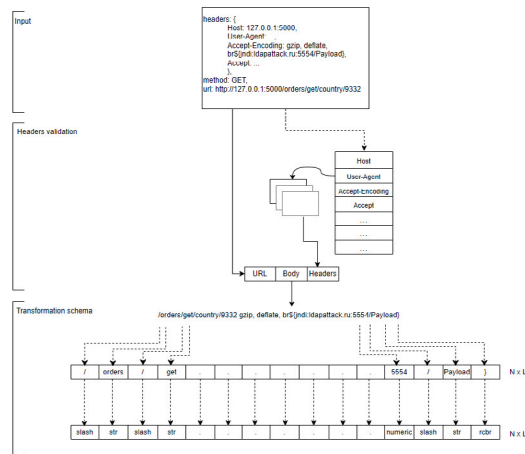
The ANN search returns a set of IDs representing neighbors' points and the similarity score between the given point and its ID. Max distance scaling is employed to scale the ANN similarity score within the given range. For every score, the maximum value gets transformed into a 0, and every other value is divided by the maximum similarity score in the range and then subtracted from 1. We use this method to invert the relationship between the original and the normalized scores to emphasize higher scores for smaller values. As in the cosine space, a smaller distance indicates that the two points are closer to each other. Let X be the similarity score, the normalized score X' is: $X' = 1 - \left(\frac{X}{\max(X)} \right)$

Lastly, we suggest an adaptive search for the best threshold for each API endpoint. As part of the detection model training, the model iteratively evaluates thresholds between 0 and 1, with increments of 0.1. The model considers the balance between precision and recall, as captured by the F1 score, to determine the optimal threshold value. As described in Fig. 2(b), in the detection stage, the first position score with the max normalized value is compared to the best threshold to determine anomaly.

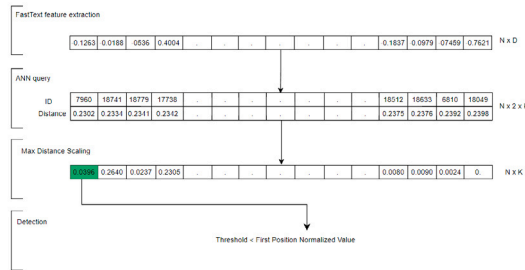
4. Experimental design

API security papers are not as prevalent as the technology itself, despite being one of the most influential technologies (Stone et al., 2022). This is particularly evident in the scarcity of ready-to-use publicly available API datasets (Torrano-Gimenez et al., 2011). For this research, we used HTTP datasets which reflect REST API traffic. REST API interactions are typically carried out over HTTP, and the resulting HTTP requests and responses are logged by the web server (Pautasso and Wilde, 2010). Several of these datasets, such as ECML/PKDD 2007 (Ware, 2007), are outdated and unsuitable for usage, with some lacking traffic diversity and volume, and others failing to encompass a variety of attacks. Similarly, the CSE-CIC-IDS 2018 (UNB, 2018) dataset contains only a small portion of web attacks and directs normal traffic requests to various web applications. Additionally, it derives anomalous traffic from DVWA application routes, complicating endpoint isolation, and often lacks textual payloads, making it unsuitable for our research. Consequently, researchers resort to creating customized datasets by primarily utilizing open-source vulnerable web applications like DVWA, BWAPP, and Mutillidae, and employ automated penetration tools such as SQLMAP (Damele and Stampar, 2015), SQLNINJA (Icesurfer and Nico, 2007), and OWASP ZAP (Bennetts, 2013) to gather malicious payloads, including the Fwaf project (Faizan, 2017).

Therefore, this research is evaluated on two datasets: CSIC 2010 (Giménez et al., 2010) and ATRDF 2023 (Lavian et al., 2023). We split both datasets into two segments: the training portion, which included normal requests and was exclusively utilized for representation learning, and the test portion, which comprised both normal and anomalous



(a) Data pre-processing step. Begins by extracting the raw components of the API request, including headers, URL, and body content. The headers are then validated to ensure they meet predefined rules, such as checking for a valid User-Agent in compliance with HTTP standards. The valid components are excluded from the subsequent stages of the process. Upon validation, the tokens are split based on the occurrence of specific ASCII characters. Each token is then transformed according to a designed schema. For example, the character "/" is converted to the token "slash," and any alphabetic word is converted to "str."



(b) Features extraction and classification-by-retrieval. In the first stage, FastText embeddings are extracted from all the API requests (N), transforming them into a vector representation with a word vector size of D (In this experiment, 100 dimensions were used). Next, an ANN query is performed to retrieve the k nearest neighbors for each API request, resulting in a matrix of neighbor IDs and corresponding cosine distance scores ($N \times K$). The distances are then normalized using max distance scaling to adjust the score range, where higher normalized values correspond to more similar requests. Finally, the system performs anomaly detection by comparing the normalized distance of the closest neighbor to an adaptive threshold, flagging API requests that fall below this threshold as anomalies.

Fig. 2. An example of the FT-ANN framework process. (a) Data pre-processing step. (b) Feature extraction and classification-by-retrieval steps.

traffic that was encoded by the model and employed for detection. The dimensions of each dataset are indicated in Table 3. The HTTP CSIC 2010 dataset (Giménez et al., 2010) is widely used (Sánchez et al., 2021; Seyyar et al., 2022; Dawadi et al., 2023; Gniewkowski et al., 2021; Mac et al., 2018; Wang et al., 2018; Moradi Vartouni et al.,

Table 3

Dataset divisions. Each dataset was split into training and testing portions, with only the normal portion used for classifier training. Tranco dataset was specifically used to train the vectorizer (e.g., FastText).

Dataset	Train		Test	
	Normal	Abnormal	Normal	Abnormal
CSIC 2010	36,000	0	36,000	24,603
ATRDF 2023	43,200	0	10,800	78,000
Tranco	1,061,095	–	–	–

2019; Jemal et al., 2020; Niu and Li, 2020; Yu et al., 2020; Ito and Iyatomi, 2018; Jemal et al., 2021b; Yan and Xiong, 2020) in the field of malicious web traffic detection. This dataset was created by the Spanish Research National Council (CSIC). It is a sample of the traffic occurring on the Spanish e-commerce web application. The dataset includes attacks such as SQLi (Fig. 3(a)), buffer overflow, information gathering, file disclosure, CRLF Injection, XSS, static attacks, and unintentional illegal requests. While unintentional illegal requests lack malicious intent, they deviate from the typical behavior of the web application and exhibit a different structure compared to regular parameter values. For instance, as shown in Fig. 3(b), an invalid DNI (Spanish national ID number) was marked as an anomaly. We divided the dataset into two segments: the training portion, which included 36,000 normal requests and was exclusively utilized for representation learning, and the test portion, which comprised both 36,000 normal and 24,603 abnormal traffic that was encoded by the model and employed for detection. It has 38 different endpoints, 8 of which have no normal representation and were excluded from our experiment.

The API Traffic Research Dataset Framework (ATRDF) (Lavian et al., 2023) is a recently published HTTP dataset publicly available which includes 18 different API endpoints. The dataset includes attacks such as Directory Traversal, Cookie Injection, Log4j (Fig. 3(c)), RCE, Log Forging, SQLi, and XSS. The dataset contains 54,000 normal and 78,000 abnormal sets of requests and responses. The training set represents 80% of the normal samples, with the remaining 20% and the abnormal samples used for testing.

In response to the unavailability of a publicly accessible pre-trained model specialized for the API domain, a generic language model was developed to establish a reliable baseline for various detection models. However, developing a robust language model necessitates a significant amount of training data. To address this requirement, an extensive dataset of 1,061,095 API examples was collected. This dataset was obtained by performing a comprehensive data collection process, involving the crawling of a random sample of websites from the Tranco top websites list (Le Pochat et al., 2019). The Tranco list is an invaluable resource for cyber-security research as it provides a publicly available compilation of the top one million most popular domains, ranked based on a combination of four reputable lists: Alexa, Cisco Umbrella, Majestic, and Quantcast. Additionally, the Tranco list offers the advantage of filtering out unavailable or malicious domains, making it a valuable asset for our research.

5. Evaluation metrics

In the realm of anomaly detection, it is commonplace to integrate a binary classification layer into the model architecture. This is due to the fundamental objective of distinguishing between normal and abnormal instances. To evaluate our architecture, we suggest using several performance measures in various experiments, including precision, recall, and F1-score.

- **Precision:** evaluates the accuracy of the positive predictions using the ratio of correctly predicted positive instances out of all instances predicted as positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

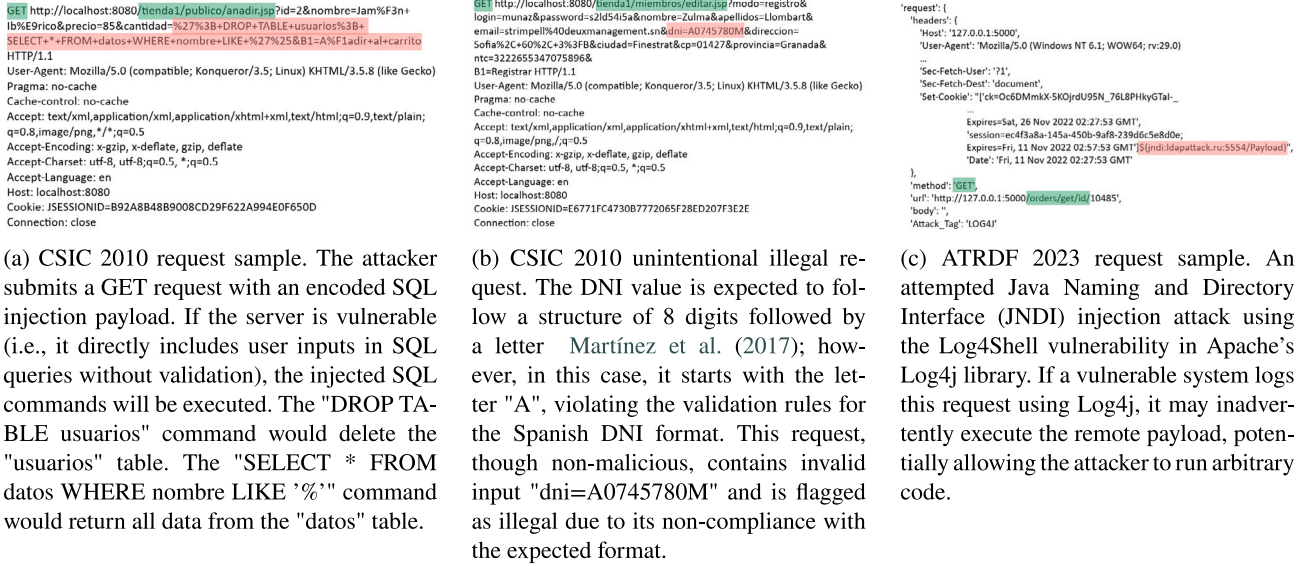


Fig. 3. An example of anomaly requests from CSIC 2010 and ATRDF 2023 datasets where endpoint definition is marked in green, malicious payload is marked in red (see Martínez et al. (2017)).

- **Recall:** measures the proportion of actual positives identified correctly.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score:** reflects the relationship between accuracy and recall.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

TP, FP, TN, and FN stand for true positive, false positive, true negative, and false negative, respectively. We also compared the training and inference times of the anomaly detection SOTAs and the queries per second (QPS) of the ANN algorithms.

6. Experimental results

In this section, we begin with a dimensionality reduction technique for data visualization, followed by a comparison with current unsupervised SOTA methods. Furthermore, we analyze the results obtained from twelve anomaly detection baseline models. Then, we conduct an ANN benchmark to explore optimization opportunities for the classification-by-retrial step. Following this, we perform an ablation study of the NN search hyper-parameter to evaluate the algorithm's sensitivity.

To gain a deeper comprehension of how the embedding layer influences the detection outcome, we assessed all anomaly detection baseline models and the ANN algorithms using two additional prominent language models: BERT (Kenton and Toutanova, 2019), and RoBERTa (Liu et al., 2019). Each model was individually trained from scratch and subsequently subjected to evaluation. We employed the RoBERTa model, specifically the RoBERTaForMaskedLM class, with a language modeling head on top.³ This model was trained with a maximum sequence length of 512, utilizing 12 hidden layers and 12 attention heads. The training process spanned 10 epochs, employing a batch size of 16, which aligns with a similar approach in a prior study by Gniewkowski et al. (2021). Likewise, we employed the BERT model, specifically BertForMaskedLM, which also incorporates a language modeling head on top.⁴ This model underwent training with

a maximum sequence length of 512, utilizing 4 hidden layers and 4 attention heads, as was the case in a similar task outlined in Guo et al. (2021).

We used Intel(R) Xeon(R) Silver 4214R CPU @ 2.40 GHz to evaluate the effectiveness of each technique. Each endpoint was evaluated individually, and we calculated the average score across all endpoints to measure the overall performance of the detector. Detecting anomalies in real-time is crucial for mitigating the impact of an attack. Therefore, we evaluate model performance by calculating the F1 score, precision, recall, and durations for training and testing in seconds.

6.1. Visualization

We use two dimension reduction algorithms, t-distributed Stochastic Neighbor Embedding (t-SNE) and Uniform Manifold Approximation and Projection (UMAP), to discover the resulting clusters or distributions of the data. The t-SNE visualization is a non-linear technique that attempts to minimize the divergence between probability distributions of data points in the high-dimensional space and their corresponding low-dimensional embedding, preserving local relationships. The UMAP visualization produces a more globally stable embedding, preserving both local and global structure (Wang et al., 2021). The visualizations shown in Fig. 4 indicate that similar requests generally tend to be grouped, enabling the exploration of a neighborhood for any given sample. Close records from different classes suggest that certain requests are quite similar, causing the embedding to overlap between the two classes. The result for ATRDF 2023 shows no overlap and clear separation between the classes while for CSIC 2010, some records from the two classes were found to be similar. We identified that most of the anomalous requests which overlap with normal requests have no malicious payload and are categorized as unintentional illegal requests.

6.2. SOTA

Our objective is to improve upon previous unsupervised SOTA techniques, as naturally, supervised methods rely on labeled data. As seen in Table 4, FT-ANN demonstrates superior accuracy compared with current SOTA unsupervised learning models on the CSIC 2010 dataset, specifically AE variants (Mac et al., 2018; Demirel and Sandikkaya, 2023; Vartouni et al., 2018; Moradi Vartouni et al., 2019), SVDD (Vartouni et al., 2023) and LLMs (Karlsen et al., 2024).

³ https://huggingface.co/docs/transformers/model_doc/RoBERTa

⁴ https://huggingface.co/docs/transformers/model_doc/bert

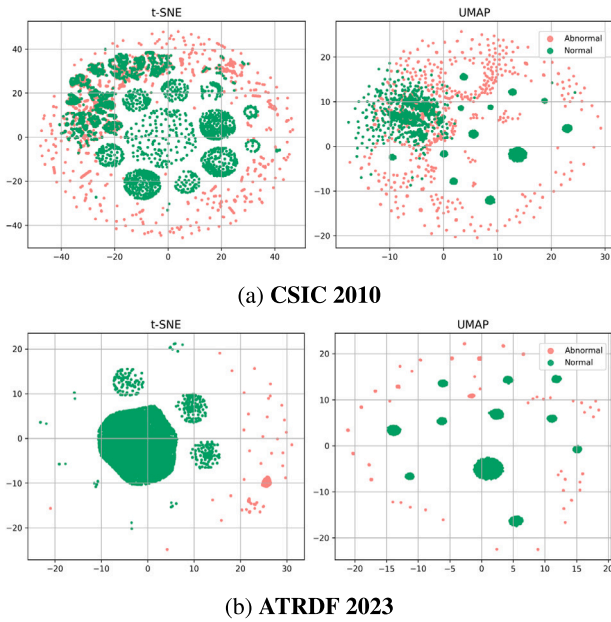


Fig. 4. Visualization of FastText embedding using t-SNE and UMAP shows the planar representation of the internal high-dimensional organization of the two classes. Each class is labeled with a unique color for clarity. Dimensionality reduction techniques, such as t-SNE, arrange data points so that those close together represent requests that FastText perceives as having similar patterns.

Table 4

Evaluation of CSIC 2010 dataset with various unsupervised methods. *The information was not provided by the author.

Method	F1 Score	Precision	Recall
FT-ANN	0.9713	0.9538	0.9954
Mac et al. (2018)	0.9463	0.9464	0.9462
Demirel and Sandikkaya (2023)	0.9274	0.9352	0.9333
Karlsen et al. (2024)	0.9022	—*	—*
Vartouni et al. (2023)	0.89	0.8583	0.9243
Vartouni et al. (2018)	0.8412	0.8029	0.8832
Moradi Vartouni et al. (2019)	0.8196	0.8122	0.8273

6.3. Baselines

We compare our method with twelve detecting outlying objects in multivariate data baseline models (Zhao et al., 2019): AE, Feature Bagging (FB), Histogram-based Outlier Detection (HBOS), IF, LOF, Minimum Covariance Determinant (MCD), One-class SVM (OCSVM), Principal Component Analysis (PCA), Deep One-Class Classification for outlier detection (DeepSVDD), Clustering-Based Local Outlier Factor (CBLOF), Kernel Density Estimation (KDE), Linear Model Deviation-based Outlier Detection (LMDD). We apply default hyper-parameters as provided by the original source code across all models for consistency and use the same pre-processed dataset to facilitate optimal comparison.

The results for the CSIC 2010 dataset, as shown in Table 5, indicate that FT-ANN and PCA have the shortest training times, at 0.0552 and 0.0575 s, respectively, while AE and LMDD are notably slower. CBLOF has a relatively fast test time but does not support incremental learning, requiring access to the entire dataset to fit the model. FT-ANN achieves the highest F1 score of 0.9713, indicating its balanced precision–recall performance. LMDD appears to exhibit the comparatively weakest performance across multiple metrics. This could be attributed to its reliance on a dissimilarity function that may not be well-suited to the complex and diverse anomalies present (Arning et al., 1996).

Several observations stand out when comparing the framework performance using BERT and RoBERTa embedding to FastText. Notably, BERT and RoBERTa result in longer training and testing times for all models. For instance, FT-ANN with FastText is approximately 87%–90% faster in training than RoBERTa and BERT, and similarly, around 78% faster in testing. This increase in time could be attributed to the more computational complexity nature of BERT (Yu et al., 2022; Xin et al., 2021) and RoBERTa (Rücklé et al., 2020) models. In terms of performance, when considering both RoBERTa and BERT embedding, FT-ANN continues to exhibit robust performance, achieving an F1 score of 0.9664 and 0.9675, respectively. This result suggests that the framework effectively leverages the contextual information embedded within the two language model representations to identify anomalies. The high precision and recall values for FT-ANN across all language models further support the framework’s ability to maintain a fine balance between detecting true anomalies and minimizing false positives. MCD with RoBERTa embedding achieves a recall of 0.9985, indicating better performance in correctly identifying positive instances compared to FT-ANN with RoBERTa embedding, which achieved 0.9972. Another noteworthy observation lies in the performance of the FT-ANN framework when using RoBERTa embedding. Surprisingly, while RoBERTa is considered an even more advanced and powerful language model compared to BERT, the F1 score for FT-ANN drops slightly to 0.9664. This outcome raises questions about why the transition to RoBERTa, which typically exhibits superior performance across a range of natural language processing tasks (Liu et al., 2019; Tarunesh et al., 2021; Rajapaksha et al., 2021), did not lead to an improved performance for this specific outlier detection method. MCD, KDE, OCSVM, FB, LOF, and LMDD exhibit consistent or improved F1 scores with BERT. However, for DeepSVDD, the F1 scores drop slightly with BERT to 0.9187 and even more with RoBERTa to 0.8999.

The results for the ATRDF 2023 dataset, as can be seen from Table 6, show that FT-ANN consistently achieves perfect scores across all metrics (F1 score, precision, recall) with BERT, FastText, and RoBERTa embedding. Comparatively, most of the baseline models also achieved high accuracy but failed to classify correctly all positive instances as reflected in the precision. DeepSVDD, AE, and IF perform well but exhibit longer build and test times. LMDD has higher build and test times, indicating again less efficient scalability.

6.4. ANN benchmark

This subsection explores FT-ANN optimization for the classification-by-retrieval step, providing a comparative benchmark for various NN algorithms. Given the diversity of ANN implementations and hyper-parameters that directly impact search result quality, it is crucial to select methods that best align with specific objectives. Although we demonstrate the effectiveness of FT-ANN using the Nmslib implementation, this module can be seamlessly replaced with other NN approaches based on specific needs and preferences.

The benchmark involved generating algorithm instances based on a configuration file written in YAML format that defines the different methods and algorithms. At the top level, the point type is specified, followed by the distance metric, and finally, each algorithm implementation to be evaluated. Each implementation specifies the Python library, and additional entries provide the necessary arguments. For clarity, an illustrative example of this configuration file is presented in Fig. 5, while the complete file is available in the project GitHub repository (Aharon et al., 2024). Both the “space” and “run_groups” lists encompass arguments that should be included at the beginning of every invocation. Each algorithm defines one or more “run_groups”, each of which is expanded into several lists of arguments. The Cartesian product of these entries results in numerous argument lists. For instance, consider the Hnswlib entry depicted in Fig. 5. This expands into three distinct algorithm instances: Cosine (Cosine Similarity), L2

Table 5

Performance evaluations of FT-ANN on the CSIC 2010 dataset in comparison to baseline models for detecting outlying objects in multivariate data.

	F1 score			Precision			Recall			Train time (s)			Test time (s)		
	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa
FT-ANN	0.9675	0.9713	0.9664	0.947	0.9538	0.9465	0.9982	0.9954	0.9972	0.4458	0.0552	0.5626	0.0353	0.0075	0.0347
MCD	0.9622	0.9572	0.9517	0.9377	0.9382	0.9159	0.9971	0.9774	0.9985	1269.2	6.3603	1261.2	2.4071	0.0021	2.5126
CBLOF	0.9404	0.9445	0.9365	0.9297	0.9318	0.9276	0.9531	0.9589	0.9477	2.8296	0.5747	2.7095	0.0245	0.001	0.0259
IF	0.9397	0.9419	0.9362	0.9289	0.9323	0.9288	0.9525	0.9538	0.9464	16.066	0.5734	15.979	0.3801	0.048	0.4007
HBOS	0.9383	0.9393	0.9367	0.9289	0.9311	0.9274	0.9501	0.9502	0.9483	0.9967	0.8365	1.0126	0.0587	0.0159	0.0613
PCA	0.938	0.9391	0.9349	0.929	0.9304	0.9271	0.9494	0.9504	0.9454	27.829	0.0574	28.412	2.635	0.0025	2.7356
AE	0.938	0.9389	0.9349	0.929	0.9303	0.9271	0.9494	0.9502	0.9454	82.872	75.0644	84.402	0.0922	0.0668	0.0978
KDE	0.937	0.9361	0.9338	0.9283	0.9296	0.9263	0.9483	0.946	0.9441	86.800	3.671	83.525	8.6993	0.3875	8.9307
OCSVM	0.9368	0.936	0.9326	0.9283	0.9295	0.9255	0.948	0.9459	0.9426	45.390	1.6392	50.025	2.8314	0.0848	3.4049
FB	0.9355	0.9354	0.9319	0.9279	0.9303	0.9255	0.946	0.9444	0.9415	10.875	10.5476	10.775	1.199	0.9365	1.2491
LOF	0.9354	0.935	0.9317	0.9279	0.9301	0.9255	0.9459	0.9439	0.9412	1.1774	1.0486	1.1146	0.1291	0.0909	0.1295
DeepSVDD	0.9187	0.9235	0.8999	0.923	0.9381	0.9155	0.921	0.9126	0.8954	38.213	32.7192	38.2421	0.0728	0.0486	0.0679
LMDD	0.3292	0.3017	0.3101	0.5701	0.528	0.5885	0.2476	0.2293	0.225	11882	225.87	11229	517.69	11.642	471.22

Table 6

Performance evaluations of FT-ANN on the ATRDF 2023 dataset in comparison to baseline models for detecting outlying objects in multivariate data.

	F1 score			Precision			Recall			Train time (s)			Test time (s)		
	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa	BERT	FastText	RoBERTa
FT-ANN	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0039	0.0029	0.0042	0.0012	0.0004	0.0012
DeepSVDD	0.9953	0.996	0.9944	0.9908	0.9921	0.9889	1.0	1.0	1.0	3.4848	3.186	3.2811	0.0383	0.0332	0.0476
AE	0.9951	0.9946	0.9951	0.9903	0.9893	0.9903	1.0	1.0	1.0	5.043	6.0278	4.9394	0.0421	0.0512	0.0412
FB	0.9944	0.9946	0.9944	0.9888	0.9893	0.9888	1.0	1.0	1.0	0.0207	0.0144	0.021	0.0139	0.0055	0.0146
IF	0.9944	0.9946	0.9944	0.9888	0.9893	0.9888	1.0	1.0	1.0	0.2578	0.1915	0.2597	0.0649	0.0307	0.0651
KDE	0.9947	0.9946	0.9947	0.9896	0.9893	0.9896	1.0	1.0	1.0	0.0043	0.001	0.0043	0.0068	0.0005	0.0069
LOF	0.9944	0.9946	0.9944	0.9888	0.9893	0.9888	1.0	1.0	1.0	0.0015	0.0014	0.0016	0.0015	0.0006	0.0011
OCSVM	0.9939	0.9946	0.9941	0.9879	0.9893	0.9884	1.0	1.0	1.0	0.0028	0.0011	0.0028	0.0017	0.0002	0.0016
PCA	0.9951	0.9946	0.9951	0.9903	0.9893	0.9903	1.0	1.0	1.0	0.0132	0.0014	0.014	0.0026	0.0002	0.0028
HBOS	0.9938	0.9944	0.9947	0.9878	0.9888	0.9896	1.0	1.0	1.0	0.4646	0.9126	0.4715	0.005	0.0228	0.0049
MCD	0.9939	0.9942	0.9886	0.9879	0.9885	0.9887	1.0	1.0	0.9898	151.32	3.618	174.05	0.2682	0.0005	0.3775
CBLOF	0.9949	0.9896	0.9957	0.99	0.9795	0.9916	1.0	1.0	1.0	0.0672	0.4408	0.0609	0.002	0.0006	0.002
LMDD	0.5641	0.292	0.6261	1.0	0.8237	1.0	0.3982	0.1825	0.4628	0.3028	0.3387	0.3208	1.2767	0.068	1.3278

(Squared L2), and IP (Inner product). Each of these instances undergoes training before being utilized for various experiments. Initially, experiments are conducted with different values of k , representing the number of neighbors to return (e.g., [10, 50, 100, 300, 400, 500, 1000, 2000, 2500, 3000]). Subsequently, experiments are conducted with varying $ef_construction$ values, which denote the size of the dynamic list used during index construction. A larger $ef_construction$ value indicates a higher quality index but also results in longer build times (e.g., [10, 20, 40, 80, 120, 200, 400, 600, 800]). Throughout each run, pertinent information is recorded, including the algorithm name, the time taken to construct the data structure used for indexing, and the outcomes of every query. These query outcomes encompass the neighboring points returned by the algorithm, the duration required to locate these neighbors, and the proximity between the neighbors and the query point. We leveraged the query results to compute the corresponding confusion matrix, enabling us to thoroughly evaluate the classification-by-retrieval performance of each algorithm. We conducted our benchmarking analysis by assessing the performance of various algorithm implementations, including Nmslib, Hnswlib, Brute-force Blas, BallTree, KDtree, CKDtree, Annoy, Faiss, and RPFforest, all of which were evaluated using the publicly accessible ANN-Benchmarks tool as a reference framework (Aumüller et al., 2020).

In the context of ANN search, selecting the optimal value for k involves determining the number of NNs to consider for predictions. The model performance evaluation and the choice of the optimized value of k rely on the F1 score, which provides a balanced assessment of precision and recall. By systematically iterating through a range of k values and assessing the F1 score for each value, we can identify the k value that yields the highest F1 score and use it in our final endpoint evaluation. During this experiment, the performance of the model was validated by iteratively testing different values of k , ranging from 1 to 1000.

One of the most important challenges in any cybersecurity system is to minimize FP and take action in real-time (Kabir and Hartmann,

```

- name: Hnswlib
  library: Hnswlib
  method: [Hnswlib]
  space: [cosine,l2,ip]
  run_groups:
    K:
      query_args: [[10,50,100,300,400,
        500,1000,2000,2500,3000]]
    ef_construction:
      query_args: [[10, 20, 40, 80,
        120, 200, 400, 600, 800]]

```

Fig. 5. Example configuration for the Hnswlib algorithm.

2018). Therefore, we conducted a trade-off analysis between QPS and index build time (training) with precision. The benchmark results for the CSIC 2010 dataset despite in Figs. 6 and 7. We show only the leading algorithms for clarity, while the complete file is available in the project GitHub repository (Aharon et al., 2024).

The results show that FT-ANN (Nmslib-COS) offers a compelling balance of speed and accuracy, with a QPS of 4795.43 and a precision of 0.9498, FT-ANN excels in delivering fast query responses while maintaining competitive precision. Hnswlib-IP achieves the highest QPS (6,065.70) and slightly higher precision (0.9547), and Nmslib-L1 and Nmslib-L2 also show strong performance with high QPS and slightly higher precision. Algorithms like Brute-force-Blas-Cosine and Faiss, despite their highest precision, significantly lag in QPS. When considering build time, FT-ANN demonstrates a moderate build time of 0.0519 s. While some algorithms like Brute-force-Blas variants have significantly shorter build times (e.g., Brute-force-Blas-Cityblock with 0.0004 s), they

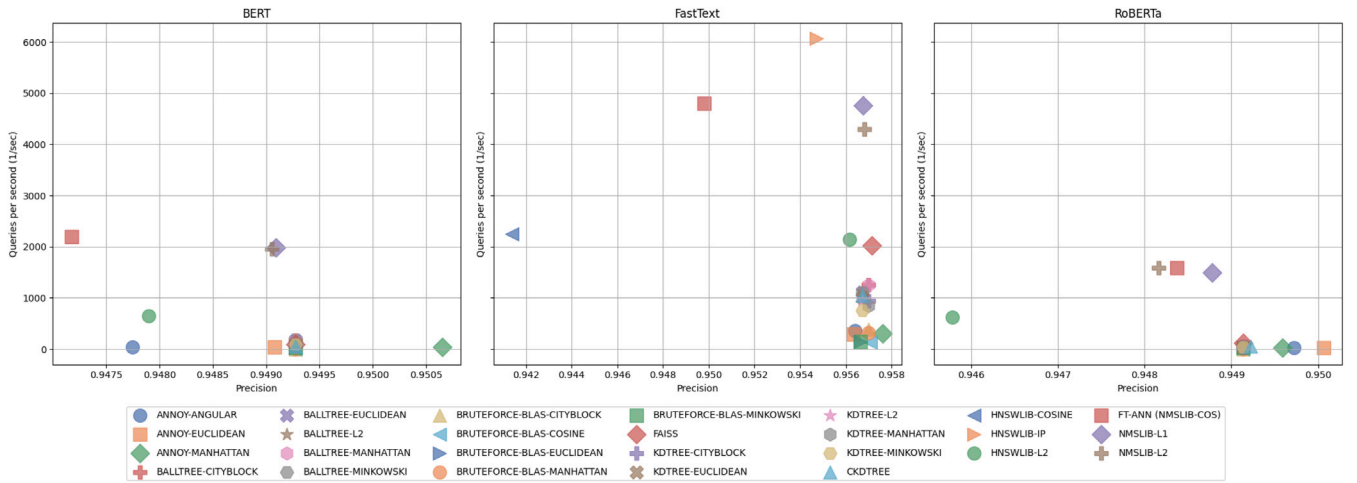


Fig. 6. Precision-QPS (queries per second) trade-off for top-performing in-memory ANN algorithms on the CSIC 2010 dataset — higher and to the right indicates better performance.

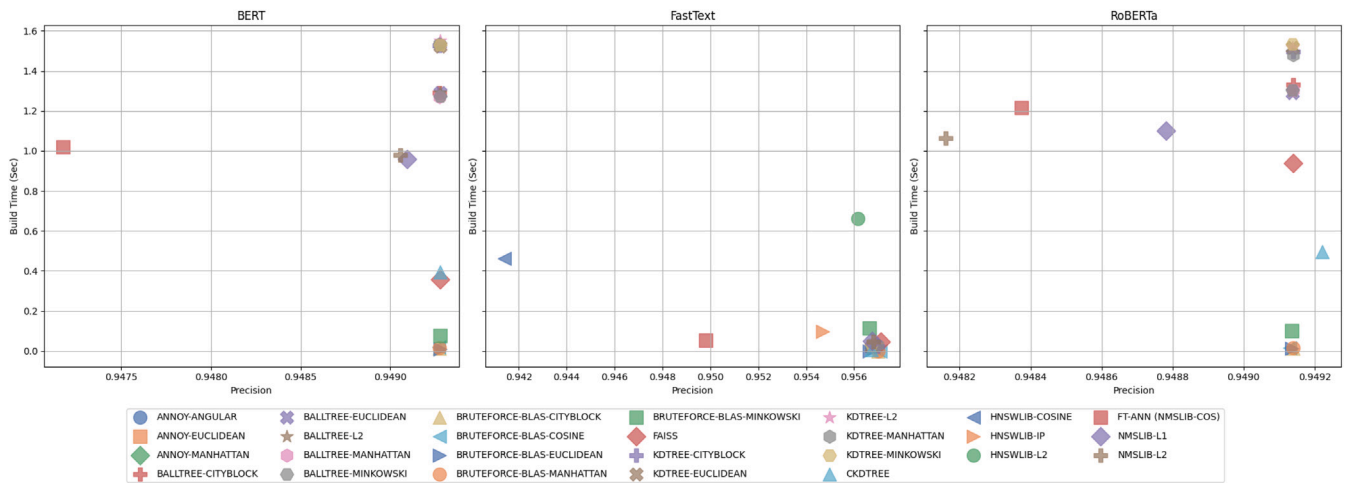


Fig. 7. Precision-Index build time trade-off in seconds for top-performing in-memory ANN algorithms on the CSIC 2010 dataset — lower and to the right indicates better performance.

lag far behind in QPS. On the other hand, algorithms like Hnswlib-Cosine and Hnswlib-L2, despite achieving relatively high QPS, have longer build times (0.461 and 0.660 s, respectively). The Annoy algorithm's slower build time and lower QPS may be due to generating random projections and building binary trees for its search index, which requires significant computation to maintain data relationships within projection dimensions, leading to slightly longer training times compared to other models (Cheng et al., 2021). The results with BERT and RoBERTa embedding show significant differences in the performance metrics. FT-ANN with BERT achieves a precision of 0.9472, a QPS of 2,189.39, and a build time of 1.0195 s, while with RoBERTa, it attains a precision of 0.9484, a QPS of 1,591.54, and a build time of 1.2153 s. Other algorithms like Hnswlib-IP and Nmslib-L1 show decreased QPS and longer build times with BERT and RoBERTa embedding, reflecting the increased computational complexity and processing requirements of these advanced models. Despite maintaining high precision, algorithms such as Faiss and BruteForce-Blas variants exhibit significantly lower QPS, highlighting their limitations in real-time applications. These insights underline that while BERT and RoBERTa embeddings provide high precision, they often come at the cost of reduced speed and increased computational demands compared to FastText.

Most ANN algorithms perform excellently on ATRDF 2023, so we chose to focus on comparing QPS and build time across different methods to evaluate their speed and scalability. The results are shown

in Figs. 8 and 9. FT-ANN using FastText embedding demonstrates a strong balance with high QPS (10,729.09) and reasonable build time (0.002997 s). In comparison, Nmslib-L1 and Nmslib-L2 outperform FT-ANN in terms of QPS, with values of 11,863.19 and 11,500.90, respectively, and slightly lower build times. Faiss and Hnswlib variants also offer high QPS with minimal build time. In contrast, BallTree variants achieve modest QPS with extremely low build times. BruteForce-Blas models, despite their simplicity and extremely low build times, achieve lower QPS, making them less suitable for high-throughput applications but beneficial where build time is critical. Generally, we observe the consistency in the model's performance relative to each other under the different language model limitations.

To gain a better understanding of how hyper-parameters for NN search impact the performance of our framework, we conducted an ablation study. This study specifically examines how varying the parameter k which denotes the number of NNs to retrieve or consider for each query point. Generally, small k values make the model highly sensitive to local patterns, allowing it to focus more closely on nearby points. However, this can also make the model more sensitive to noise. Larger values lead to a smoother decision boundary, considering a broader area, which can make the search or classification less sensitive to noise but more generalized. In Fig. 10 we show the analysis of the CSIC 2010 dataset for selected algorithms, the complete analysis is available in the project's GitHub repository (Aharon et al., 2024).

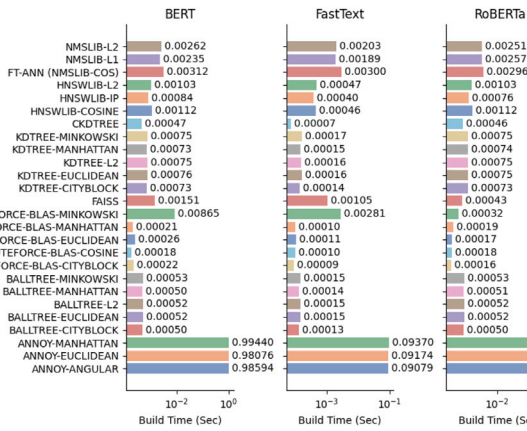


Fig. 8. Evaluation of Index Build Time (seconds) for In-Memory ANN Algorithms on the ATRDF 2023 Dataset. The Y-axis represents algorithm names, and the X-axis shows build time in log scale.

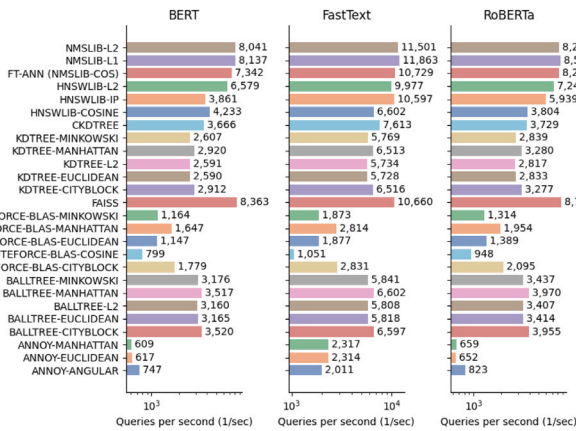


Fig. 9. Evaluation of QPS (queries per second) for In-Memory ANN Algorithms on the ATRDF 2023 Dataset. The Y-axis represents algorithm names, and the X-axis shows QPS in log scale.

The results highlight that for Hnswlib-Cosine, increasing the value of k improves precision up to 300 NNs. Similarly, a slight increase in precision is observed for Nmslib-L1, Nmslib-L2, and Annoy-Manhattan up to 50 NNs. Interestingly, Annoy-Manhattan result improves with 2000 NNs, suggesting that a higher number of neighbors allows the approximation to better capture the underlying structure of the data. Nmslib-Cosine, on the other hand, shows relatively stable precision across all k values, suggesting that even a minimal number of neighbors per query is sufficient to maximize its performance. This indicates that the algorithm is not significantly affected by the number of NNs and is less sensitive to variations in k compared to the other methods.

7. Discussion and conclusions

This paper suggests an innovative unsupervised few-shot anomaly detection framework that leverages a dedicated generic language model for API based on FastText embedding and uses ANN search in a Classification-by-retrieval approach. We demonstrated that API Injection attacks can be identified using only a few examples of normal API requests. Our framework not only accurately classifies the API traffic but also does it quickly and effectively with the support of incremental learning by the ANN, which is crucial for real-world systems. To the best of our knowledge, this is the first work to utilize a Classification-by-retrieval framework based on the generalized approach of FastText embedding combined with the approximate search

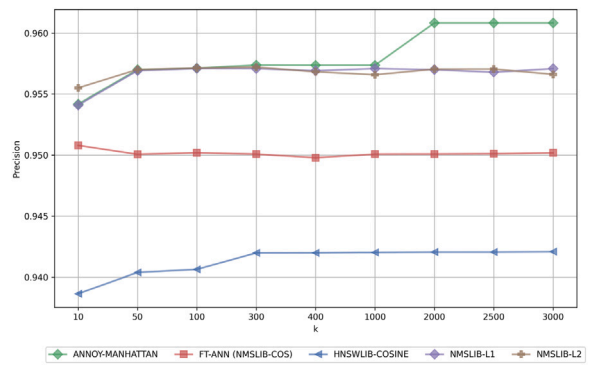


Fig. 10. Average precision across endpoints for different k parameter values on the CSIC 2010 dataset for selected in-memory ANN algorithms.

to find anomalies in API traffic. Additionally, we presented a unique pre-processing technique to enhance input generalization and simplify API structure. This approach encompasses dividing input data into individual tokens and then constraining the vocabulary to a limited set of tokens. Consequently, the API structure becomes streamlined as the number of unique tokens diminishes, enabling input generalization and achieving high detection accuracy even with minimal examples per class. We presented several baseline anomaly detection models for this objective through two other widely adopted language models, BERT and RoBERTa. We compared our framework to the current SOTA unsupervised techniques and showed that it achieved the highest accuracy on the CSIC 2010 dataset.

We demonstrated how the framework can be optimized by considering different ANN algorithms and analyzed the sensitivity of each method to the parameter k in the NN search. While we observed that the cosine similarity implementation in Nmslib does not produce the highest accuracy among ANN algorithms, its straightforward implementation and seamless integration, and a good balance of QPS and build time make it a suitable choice for practical applications, particularly those like Amazon Elasticsearch Service (Amazon Web Services, 2020) that prioritize real-world compatibility.

One notable limitation of this and similar studies arises from the scarcity of up-to-date and representative network traffic datasets. Many research efforts rely solely on the CSIC 2010 dataset, raising concerns about the reliability and applicability of such data, consequently impacting the quality and generalizability of the resulting models. While our evaluation focuses on measuring the impact of FT-ANN on the HTTP dataset, it is important to acknowledge that APIs exist in diverse forms, including REST, GraphQL, gRPC, and WebSockets, each catering to specific use cases and carrying common vulnerabilities. Our future work entails expanding the evaluation to encompass a wider range of API attacks and types. Additionally, we intend to assess the stability of the framework and its resource utilization under varying loads and volumes.

CRedit authorship contribution statement

Udi Aharon: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Ran Dubin:** Writing – review & editing, Supervision, Project administration, Methodology, Conceptualization. **Amit Dvir:** Writing – review & editing, Methodology, Investigation, Conceptualization. **Chen Hajaj:** Writing – review & editing, Project administration, Methodology, Investigation, Data curation, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Udi Aharon, Ran Dubin, Amit Dvir has patent SYSTEM AND METHOD OF APPLICATION PROGRAMMING INTERFACE (API) ANOMALY DETECTION pending to Pending. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Ariel Cyber Innovation Center in conjunction with the Israel National Cyber Directorate in the Prime Minister's Office.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cose.2024.104249>.

Data availability

The dataset is publicly available.

References

- Aharon, U., Dubin, R., Dvir, A., Hajaj, C., 2024. Supplementary material: A classification-by-retrieval framework for few-shot anomaly detection to detect API injection attacks. <https://github.com/ArielCyber/FT-ANN>.
- Amazon Web Services, 2020. Build k-nearest neighbor (k-NN) similarity search engine with amazon elasticsearch service. <https://aws.amazon.com/about-aws/whats-new/2020/03/build-k-nearest-neighbor-similarity-search-engine-with-amazon-elasticsearch-service/>.
- Arning, A., Agrawal, R., Raghavan, P., 1996. A linear method for deviation detection in large databases. In: KDD, vol. 1141, (50), pp. 972–981.
- Aumüller, M., Bernhardtsson, E., Faithfull, A., 2020. ANN-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.* 87, 101374.
- Balsari, S., Fortenko, A., Blaya, J.A., Gropper, A., Jayaram, M., Matthan, R., Sahasranam, R., Shankar, M., Sarbadhikari, S.N., Bierer, B.E., et al., 2018. Reimagining health data exchange: An application programming interface-enabled roadmap for India. *J. Med. Internet Res.* 20 (7), e10725.
- Bansal, B., Srivastava, S., 2018. Sentiment classification of online consumer reviews using word vector representations. *Procedia Comput. Sci.* 132, 1147–1153.
- Battle, R., Benson, E., 2008. Bridging the semantic web and web 2.0 with representational state transfer (REST). *J. Web Semant.* 6 (1), 61–69.
- Baye, G., Hussain, F., Oracevic, A., Hussain, R., Kazmi, S.A., 2021. API security in large enterprises: Leveraging machine learning for anomaly detection. In: ISNCC. IEEE, pp. 1–6.
- BBC, 2019. Capital one data breach: Arrest after details of 106 m people stolen. <https://www.bbc.com/news/world-us-canada-49159859>, BBC.
- Bennetts, S., 2013. OWASP zed attack proxy. AppSec USA.
- Benzaid, C., Taleb, T., 2020. ZSM security: Threat surface and best practices. *IEEE Netw.* 34 (3), 124–133.
- Buchanan, W.J., Helme, S., Woodward, A., 2018. Analysis of the adoption of security headers in HTTP. *IET Inf. Secur.* 12 (2), 118–126.
- Chan, A., Kharkar, A., Moghaddam, R.Z., Mohylevskyy, Y., Helyar, A., Kamal, E., Elkamhaw, M., Sundaresan, N., 2023. Transformer-based vulnerability detection in code at EditTime: Zero-shot, few-shot, or fine-tuning? *arXiv preprint arXiv:2306.01754*.
- Chávez, E., Navarro, G., Baeza-Yates, R., Marroquín, J.L., 2001. Searching in metric spaces. *ACM Comput. Surv.* 33 (3), 273–321.
- Cheng, F., Hyndman, R.J., Panagiotelis, A., 2021. Manifold Learning With Approximate Nearest Neighbors. Technical Report, Monash University, Department of Econometrics and Business Statistics.
- Coyne, M., 2020. Zoom's big security problems summarized. <https://www.forbes.com/sites/marleycoyne/2020/04/03/zooms-big-security-problems-summarized/?sh=46fc370f4641>, Forbes.
- Damele, B., Stampar, M., 2015. SQLMAP: Automatic SQL injection and database takeover tool. <http://sqlmap.org>.
- Dawadi, B.R., Adhikari, B., Srivastava, D.K., 2023. Deep learning technique-enabled web application firewall for the detection of web attacks. *Sensors* 23 (4), 2073.
- Demirel, D.Y., Sandikkaya, M.T., 2023. ACUM: An approach to combining unsupervised methods for detecting malicious web sessions. In: 2023 8th International Conference on Computer Science and Engineering. UBMK, IEEE, pp. 288–293.
- Faizan, A., 2017. Fwaf machine learning driven web firewall. <https://github.com/faizann24/Fwaf-Machine-Learning-driven-Web-Application-Firewall>.
- Fett, D., Küsters, R., Schmitz, G., 2016. A comprehensive formal security analysis of OAuth 2.0. In: SIGSAC CCCS. pp. 1204–1215.
- Giménez, C.T., Villegas, A.P., Marañón, G.Á., 2010. HTTP data set CSIC 2010. CSIC 64.
- Gniewkowsky, M., Maciejewski, H., Surmacz, T.R., Walentynowicz, W., 2021. HTTP2vec: Embedding of HTTP requests for detection of anomalous traffic. *ArXiv*, *arXiv:2108.01763*.
- Greig, J., 2023. Hilton denies hack after data from 3.7 million honors customers offered for sale. <https://therecord.media/hilton-denies-hack-after-data-from-3-7-million-honors-customer-offered-for-sale/>, The Record.
- Guo, H., Yuan, S., Wu, X., 2021. LogBERT: Log anomaly detection via BERT. In: 2021 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8.
- Hajebi, K., Abbasi-Yadkori, Y., Shahbazi, H., Zhang, H., 2011. Fast approximate nearest-neighbor search with K-nearest neighbor graph. In: AI. pp. 1312–1317.
- Harlicaj, E., et al., 2021. Anomaly Detection of Web-Based Attacks in Microservices (Master's thesis). Aalto University.
- Hsu, Y.C., Shen, Y., Jin, H., Kira, Z., 2020. Generalized odin: Detecting out-of-distribution image without learning from out-of-distribution data. In: CVF. pp. 10951–10960.
- IBM, 2016. Innovation in the API economy: Building winning experiences and new capabilities to compete. <https://www.ibm.com/downloads/cas/OXV3LYLO>.
- Icesurfer, Nico, 2007. SQLNINJA: SQL server injection & takeover tool. <https://sqlninja.sourceforge.net>.
- Indyk, P., Motwani, R., 1998. Approximate nearest neighbors: Towards removing the curse of dimensionality. In: ACM Symposium on Theory of Computing. pp. 604–613.
- Ito, M., Iyatomi, H., 2018. Web application firewall using character-level convolutional neural network. In: CSPA. pp. 103–106.
- Jemal, I., Haddar, M.A., Cheikhrouhou, O., Mahfoudhi, A., 2020. M-CNN: a new hybrid deep learning model for web security. In: AICCSA. pp. 1–7.
- Jemal, I., Haddar, M.A., Cheikhrouhou, O., Mahfoudhi, A., 2021a. Performance evaluation of convolutional neural network for web security. *Comput. Commun.* 175, 58–67.
- Jemal, I., Haddar, M.A., Cheikhrouhou, O., Mahfoudhi, A., 2021b. Performance evaluation of convolutional neural network for web security. *Comput. Commun.* 175, 58–67.
- Joulin, A., Grave, E., Bojanowski, P., Mikolov, T., 2017. Bag of tricks for efficient text classification. In: Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers. Association for Computational Linguistics, pp. 427–431.
- Kabir, M.F., Hartmann, S., 2018. Cyber security challenges: An efficient intrusion detection system design. In: 2018 International Young Engineers Forum. YEF-ECE, IEEE, pp. 19–24.
- Karlsen, E., Luo, X., Zincir-Heywood, N., Heywood, M., 2024. Large language models and unsupervised feature learning: Implications for log analysis. *Ann. Telecommun.* 1–19.
- Kenton, J.D.M.W.C., Toutanova, L.K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of NaaCL-HLT, vol. 1, Minneapolis, Minnesota, p. 2.
- Lavian, S., Dubin, R., Dvir, A., 2023. The API traffic research dataset framework (ATRDF). https://github.com/ArielCyber/Cisco_Ariel_Uni_API_security_challenge.
- Le Pochat, V., Van Goethem, T., Tajalizadehkhoob, S., Korczyński, M., Joosen, W., 2019. Tranco: A research-oriented top sites ranking hardened against manipulation. In: 26th Annual Network and Distributed System Security Symposium (NDSS 2019).
- Li, B., Han, L., 2013. Distance weighted cosine similarity measure for text classification. In: IDEAL, China, Oct.. pp. 611–618.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Mac, H., Truong, D., Nguyen, L., Nguyen, H., Tran, H.A., Tran, D., 2018. Detecting attacks on web applications using autoencoder. In: ICT. pp. 416–421.
- Malkov, Y.A., Yashunin, D.A., 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42 (4), 824–836.
- Martínez, V.G., Encinas, L.H., Muñoz, A.M., Mariño, M.Á., Guardado, D.A., 2017. A comparative study of three spanish egovernment smart cards. *Log. J. IGPL* 25 (1), 42–53.

- Mendoza, A., Gu, G., 2018. Mobile application web API reconnaissance: Web-to-mobile inconsistencies & vulnerabilities. In: SP. pp. 756–769.
- Moradi Vartouni, A., Mehralian, S., Teshnehlal, M., Sedighian Kashi, S., 2019. Auto-encoder LSTM methods for anomaly-based web application firewall. *Int. J. Inf. Commun. Technol. Res.* 11 (3), 49–56.
- Naidan, B., Boytsov, L., Malkov, Y., Novak, D., 2014. Non-metric space library (NMSLIB): An efficient similarity search library and a toolkit for evaluation of k-NN methods for generic non-metric spaces. Online; <https://github.com/nmslib/nmslib>. (Accessed 14 July 2014).
- Niu, Q., Li, X., 2020. A high-performance web attack detection method based on CNN-GRU model. In: ITNEC, vol. 1, pp. 804–808.
- Ofoeda, J., Boateng, R., Effah, J., 2019. Application programming interface (API) research: A review of the past to inform the future. *IJEIS* 15 (3), 76–95.
- Ombagi, J., 2017. Time-based blind SQL injection via HTTP headers: Fuzzing and exploitation. In: Conference: 2017 Strathmore Research Symposium, At Nairobi, Kenya.
- Pautasso, C., Wilde, E., 2010. Restful web services: Principles, patterns, emerging technologies. In: Proceedings of the 19th International Conference on World Wide Web. pp. 1359–1360.
- Ponomarenko, A., Avrelil, N., Naidan, B., Boytsov, L., 2014. Comparative analysis of data structures for approximate nearest neighbor search. In: Data Analytics. pp. 125–130.
- Qamar, A.M., Gaussier, E., Chevallet, J.P., Lim, J.H., 2008. Similarity learning for nearest neighbor classification. In: ICDM. IEEE, pp. 983–988.
- Rajapaksha, P., Farahbakhsh, R., Crespi, N., 2021. BERT, XLNet or RoBERTa: the best transfer learning model to detect clickbaits. *IEEE Access* 9, 154704–154716.
- Reddy, M., 2011. API Design for C++. Elsevier, p. 1.5.
- Reddy, A.S., Rudra, B., 2021. Evaluation of recurrent neural networks for detecting injections in API requests. In: CCWC. pp. 0936–0941. <http://dx.doi.org/10.1109/CCWC51732.2021.9376034>.
- Research, F.A., 2017. Fasttext library for efficient learning of word representations and sentence classification. Online; <https://github.com/facebookresearch/fastText/>. (Accessed 02 December 2017).
- Rücklé, A., Geigle, G., Glockner, M., Beck, T., Pfeiffer, J., Reimers, N., Gurevych, I., 2020. AdapterDrop: On the efficiency of adapters in transformers. *arXiv preprint arXiv:2010.11918*.
- Sánchez, P.M.S., Valero, J.M.J., Celdrán, A.H., Bovet, G., Pérez, M.G., Pérez, G.M., 2021. A survey on device behavior fingerprinting: Data sources, techniques, application scenarios, and datasets. *IEEE Commun. Surv. Tutor.* 23 (2), 1048–1077.
- Seyyar, Y.E., Yavuz, A.G., Ünver, H.M., 2022. An attack detection framework based on BERT and deep learning. *IEEE Access* 10, 68633–68644.
- Shen, F., Mu, Y., Yang, Y., Liu, W., Liu, L., Song, J., Shen, H.T., 2017. Classification by retrieval: Binarizing data and classifiers. In: ACM SIGIR. pp. 595–604.
- Shi, W., Michael, J., Gururangan, S., Zettlemoyer, L., 2022. Nearest neighbor zero-shot inference. *arXiv preprint arXiv:2205.13792*.
- Stone, J.E., Griffin, K.S., Amstutz, J., DeMarle, D.E., Sherman, W.R., Günther, J., 2022. ANARI: A 3-D rendering API standard. *Comput. Sci. Eng.* 24 (2), 7–18.
- Sun, R., Wang, Q., Guo, L., 2021. Research towards key issues of API security. In: CNCERT, Beijing, China, July 20–21. pp. 179–192.
- Tara Siegel, B., Tiffany, H., Nicole, P., Ron, L., 2017. Equifax says cyberattack may have affected 143 million in the U.S. *N.Y. Times* <https://www.nytimes.com/2017/09/07/business/equifax-cyberattack.html>, The New York Times.
- Tarunesh, I., Aditya, S., Choudhury, M., 2021. Trusting RoBERTa over BERT: Insights from checklisting the natural language inference task. *arXiv preprint arXiv:2107.07229*.
- Techapanurak, E., Suganuma, M., Okatani, T., 2020. Hyperparameter-free out-of-distribution detection using cosine similarity. In: ACCV.
- Torrano-Gimenez, C., Nguyen, H.T., Alvarez, G., Petrović, S., Franke, K., 2011. Applying feature selection to payload-based web application firewalls. In: IWSCN. pp. 75–81.
- UNB, 2018. A collaborative project between the communications security establishment (CSE) & the Canadian institute for cybersecurity (CIC). <https://www.unb.ca/cic/datasets/ids-2018.html>.
- Valero-Mas, J.J., Gallego, A.J., Alonso-Jiménez, P., Serra, X., 2023. Multilabel prototype generation for data reduction in k-nearest neighbour classification. *Pattern Recognit.* 135, 109190.
- Vartouni, A.M., Kashi, S.S., Teshnehlal, M., 2018. An anomaly detection method to detect web attacks using stacked auto-encoder. In: 2018 6th Iranian Joint Congress on Fuzzy and Intelligent Systems. CFIS, IEEE, pp. 131–134.
- Vartouni, A.M., Shokri, M., Teshnehlal, M., 2023. Auto-threshold deep SVDD for anomaly-based web application firewall. *Authorea Preprints*.
- Wang, Y., Huang, H., Rudin, C., Shaposhnik, Y., 2021. Understanding how dimension reduction tools work: An empirical approach to deciphering t-SNE, UMAP, TriMAP, and PaCMAP for data visualization. *J. Mach. Learn. Res.* 22 (201), 1–73.
- Wang, J., Zhou, Z., Chen, J., 2018. Evaluating CNN and LSTM for web attack detection. In: ICMLC. pp. 283–287.
- Ware, B., 2007. Analyzing web traffic ECML/PKDD 2007 discovery challenge. <https://www.lirmm.fr/pkdd2007-challenge/comites.html>.
- Xia, P., Zhang, L., Li, F., 2015. Learning similarity with cosine similarity ensemble. *Inform. Sci.* 307, 39–52.
- Xiao, L., Matsumoto, S., Ishikawa, T., Sakurai, K., 2016. SQL injection attack detection method using expectation criterion. In: 2016 Fourth International Symposium on Computing and Networking. CANDAR, IEEE, pp. 649–654.
- Xin, J., Tang, R., Yu, Y., Lin, J., 2021. Berxit: Early exiting for BERT with better fine-tuning and extension to regression. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume. pp. 91–104.
- Yan, L., Xiong, J., 2020. Web-APT-detect: a framework for web-based advanced persistent threat detection using self-translation machine with attention. *IEEE Lett. Comput. Soc.* 3 (2), 66–69.
- Yu, L., Chen, L., Dong, J., Li, M., Liu, L., Zhao, B., Zhang, C., 2020. Detecting malicious web requests using an enhanced textCNN. In: COMPSAC. IEEE, pp. 768–777.
- Yu, T., Fei, H., Li, P., 2022. U-BERT for fast and scalable text-image retrieval. In: Proceedings of the 2022 ACM SIGIR International Conference on Theory of Information Retrieval. pp. 193–203.
- Zhao, Y., Nasrullah, Z., Li, Z., 2019. Pyod: A python toolbox for scalable outlier detection. *J. Mach. Learn. Res.* 20 (96), 1–7.



Udi Aharon is currently pursuing a Ph.D. degree in the Department of Computer Science at Ariel University, Israel. His research activities span the fields of Machine Learning and Cybersecurity. Specifically, his work focuses on enhancing API security through the application of text-based models.



Dr. Ran Dubin received his B.Sc., M.Sc., and Ph.D. degrees from Ben-Gurion University, Beer Sheva, Israel, all in communication systems engineering. He is a faculty member at the Computer and Software Engineering Department, Ariel University, Israel. His research interests revolve around zero-trust cyber protection, malware disarms and reconstruction, encrypted network traffic detection, Deep Packet Inspection (DPI), bypassing AI, Natural Language Processing, and AI trust.



Prof. Amit Dvir received his B.Sc., M.Sc., and Ph.D. degrees in communication systems engineering from Ben-Gurion University, Beer Sheva, Israel. He is currently the head of the Computer and Software Engineering Department and the head of the Ariel Cyber Innovation Center at Ariel University, Israel. From 2011 to 2012, he was a Postdoctoral Fellow at the Laboratory of Cryptography and System Security, Budapest, Hungary. His research interests include enrichment data from encrypted traffic.



Dr. Chen Hajaj holds Ph.D. (Computer Science), M.Sc. (Electrical Engineering), and B.Sc. (Computer Engineering) degrees, all from Bar-Ilan University. He serves as a faculty member in the Department of Industrial Engineering and Management and a member of the Ariel Cyber Innovation Center. Following his doctoral studies, he completed a post-doctoral fellowship at Vanderbilt University (2016–2018). His research interests span Machine Learning, Game Theory, and Cybersecurity, with a particular focus on encrypted traffic classification, adversarial artificial intelligence, and multimodal classification techniques.